Assignment III

Continuous Integration and Continuous Deployment

(DSO101)

Bachelor's of Engineering in Software Engineering (SWE)

Date of submission: 29th April

RUB Wheel of Academic Law: Academic Dishonesty

Section H2 of the Royal University of Bhutan's *Wheel of Academic Law* provides the following definition of academic dishonesty:

Academic dishonesty may be defined as any attempt by a student to gain an unfair advantage in any assessment. It may be demonstrated by one of the following:

1. **Collusion:** the representation of a piece of unauthorized group work as the work of a single candidate.
2. **Commissioning:** submitting an assignment done by another person as the student's own work.
3. **Duplication**: the inclusion in coursework of material identical or substantially similar to material which has already been submitted for any other assessment within the University.
4. **False declaration**: making a false declaration in order to receive special consideration by an Examination Board or to obtain extensions to deadlines or exemption from work.
5. **Falsification of data**: presentation of data in laboratory reports, projects, etc., based on work purported to have been carried out by the student, which has been invented, altered or copied by the student.
6. **Plagiarism**: the unacknowledged use of another's work as if it were one's own.

Examples are:

- verbatim copying of another's work without acknowledgement.
- paraphrasing of another's work by simply changing a few words or altering the order of presentation, without acknowledgement.
- ideas or intellectual data in any form presented as one's own without acknowledging the source(s).
- making significant use of unattributed digital images such as graphs, tables, photographs, etc. taken from test books, articles, films, plays, handouts, internet, or any other source, whether published or unpublished.
- submission of a piece of work which has previously been assessed for a different award or module or at a different institution as if it were new work.
- use of any material without prior permission of copyright from appropriate authority or owner of the materials used".

Objective

In this assignment, you will configure a **GitHub Actions** workflow to automate:

1. **Building** a Docker container for your application.
2. **Pushing** the container to **DockerHub**.
3. **Deploying** the container on **Render.com**.

You will use the **to-do list application** from **Assignment 1** (or any Node.js application).

Tools & Technologies

Tool	Purpose
GitHub	Hosting source code
GitHub Actions	CI/CD automation
Docker	Containerization
DockerHub	Container registry
Render.com	Cloud deployment
Node.js & npm	Backend runtime & package management
Jest/Mocha	Testing framework

Tasks

Task 1:

1. Verify your Github repository setup
    a. Ensure your package.json file has relevant scripts
    b. Ensure that your repository is public

Task 2:

1. Verify the Dockerfiles in your relevant repository

Expected structure:

```
# Use Node.js LTS
FROM node:20-alpine

# Set working directory
WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy all files
COPY . .

# Run tests (optional)
RUN npm test

# Expose the app port
EXPOSE 3000

# Start the app
CMD ["npm", "start"]
```

2. Test your applications locally after containerizing them.

Task 3:

1. **Create `.github/workflows/deploy.yml`:**

Expected structure:

```
on:
  push:
      branches: ["main"]

jobs:
  build-and-deploy:
      runs-on: ubuntu-latest

      steps:
      # 1. Checkout code
      - name: Checkout Repository
      uses: actions/checkout@v4

      # 2. Login to DockerHub
      - name: Login to DockerHub
      uses: docker/login-action@v3
      with:
            username: ${{ secrets.DOCKERHUB_USERNAME }}
            password: ${{ secrets.DOCKERHUB_TOKEN }}

      # 3. Build & Push Docker Image
      - name: Build and Push Docker Image
      run: |
            docker build -t ${{ secrets.DOCKERHUB_USERNAME
}}/todo-app:latest .
            docker push ${{ secrets.DOCKERHUB_USERNAME }}/todo-app:latest

      # 4. Deploy to Render.com (via webhook)
      - name: Trigger Render Deployment
      run: ** REFER NOTE BELOW**
```

NOTE: Render does not automatically redeploy new images that are pushed to the docker hub, to automate a render redeployment, you will need to make a call to render the eployment webhook service in your GitHub actions .yaml file. Read more on this here: LINK

2. **Add GitHub Secrets:**

You will need to add relevant secrets to your GitHub account, this will depend on the nature of your docker hub repository. For ease of implementation:

1. Ensure your docker hub repository is public
2. Ensure that your render deployment tokens and webhooks are added as secrets and referenced accordingly.

NOTE: DONOT HARDCODE YOUR CREDENTIALS IN YOUR CODE

Task 4:

1. Create a new website in Render.com
2. Select deploy from the existing image
3. Follow the configuration steps as needed.

Expected outcome:

1. **GitHub Repo Link** (with `Dockerfile`, `.github/workflows/deploy.yml`).
2. **Screenshots of:**
    - Successful GitHub Actions workflow.
    - DockerHub pushed the image.
    - Render.com deployment.
3. **Short Report (`README.md`):**
    - Steps taken.
    - Challenges faced.
    - Learning outcomes.
    - Screenshots mentioned above
    - A link to your render deployment instance